

Software Track

Factory Data and Production Line

Complete this tab only if Software is your chosen track. The Introduction tab contains the authoritative deadline and submission instructions.

Goal

Build a platform that:

1. Collects factory data from several local sources.
2. Normalizes it into one traceable operational dataset.
3. Uses that dataset to show and manage production-line status.

We value a complete, reliable vertical slice more than a large number of features.

Where the requirements leave implementation choices open, document your assumptions, design decisions, and trade-offs in the README. We will assess whether your chosen approach is deterministic, internally consistent, appropriately tested, and clearly explained.

Scenario

An industrial laundry processes hotel linen through six fixed steps:

1. Receiving
2. Sorting
3. Washing
4. Drying
5. Ironing and folding
6. Packing and dispatch

Its operational data is fragmented across an internal application, a supplier portal, a production database, and optionally MQTT-enabled machines. Managers cannot see a trustworthy end-to-end view of each batch.

Your solution must improve visibility and traceability. It must not automatically optimize the schedule or control machines.

Required Local Data Sources

1. Application API

Provide a local REST fixture containing work orders, batches, receiving records, and dispatch records.

The collector must support:

- Pagination
- Request timeouts
- Retry of transient failures

2. Data Crawler

Provide a locally hosted, paginated supplier page.

Crawl:

- Delivery number
- Supplier
- Batch ID
- Quantity
- Delivery time
- A stable source-record identifier

Prevent pagination loops and report malformed rows without failing the whole collection run.

3. Database Connection

Use a locally hosted PostgreSQL or MySQL database provided through Docker Compose.

The solution must allow the user to:

- Register and verify the database connection before use
- Discover available tables and columns
- Select a production table for collection
- Collect records from the selected table

Credentials must be supplied through environment variables or a masked secret input. After registration, secrets must not be returned by the API, redisplayed in the interface, written to logs, or committed to source control.

4. MQTT — Optional

Optionally provide a local Mosquitto-based MQTT simulation and collect washing or drying telemetry from it.

MQTT may run as an optional Docker Compose service. All required functionality and tests must pass when MQTT is not implemented or enabled.

Six-Step Sample Data

All fixture records must use consistent identifiers and be joinable across sources.

Every operational record must contain a `batchId`. The Application API fixture must map each `batchId` to its `workOrderId` and `lineId`. Other sources may omit `workOrderId` or `lineId` when they can be joined reliably through `batchId`.

Fixtures must include:

- At least one duplicate source observation
- At least one late event from an earlier station
- Data covering all six production steps

Candidates must define and document a deterministic deduplication and conflict-handling policy. Original source observations must remain available for provenance and audit.

Together, the required API, crawler, and database fixtures must cover the following steps:

Step	Station code	Required source
1	RECEIVING	Supplier crawler
2	SORTING	Production database
3	WASHING	Production database; optional MQTT telemetry
4	DRYING	Production database; optional MQTT telemetry
5	FOLDING	Production database
6	DISPATCH	Application API or production database

Required Collection Workflow

The user must be able to:

1. Register and test a source.
2. Discover its available schema or fields.
3. Select what should be collected.
4. Run collection manually.
5. Inspect collection status, duration, counts, and errors.
6. Preview normalized records with their source and collection-run provenance.

Production-Line Management

Build the production view from normalized records.

For each line and station, show:

- Assigned work orders and batches
- Current station and completed quantity
- Work in progress
- Last event time and data freshness
- Stale, blocked, missing-data, and quality indicators
- Links to the contributing source records and collection run

Batch State

Evaluate batch state in the following order:

1. **COMPLETED**: an accepted **DISPATCH** event exists.
2. **BLOCKED**: no accepted **DISPATCH** event exists and a manager has applied a block that has not subsequently been resumed.
3. **IN_PROGRESS**: no active block or accepted **DISPATCH** event exists, and at least one accepted event from **RECEIVING** through **FOLDING** exists.
4. **PLANNED**: a work order exists but no accepted operational event exists.

A later-station event may place a batch in progress even when an earlier event is missing, but the batch must display a missing-data indicator.

Station Progress and WIP

Current station is the furthest station reached by an accepted canonical event, using the six-step order defined above.

A late event from an earlier station may update history but must not move a batch backwards.

Station WIP is the number of non-completed batches whose current station is that station.

Completed quantity is the quantity reported at that station after applying the candidate's documented deterministic deduplication policy. Duplicate source observations must not cause quantities to be counted more than once.

Freshness

Show the last accepted event time and the resulting data freshness.

The default stale threshold is 15 minutes and must be configurable.

Management Events

Managers must be able to:

- Acknowledge an exception
- Block a batch
- Resume a batch

- Add a note

Store these actions as append-only, organization-scoped management events containing the actor and timestamp. Do not overwrite collected source history.

Full authentication and user-management functionality is not required. Candidates may use a seeded organization and actor, but all management events must still include `organizationId`, actor, and timestamp.

Required User Interface

The application must contain the following two primary views.

Data Sources

This view must support the required source-registration, discovery, selection, collection, monitoring, error-inspection, and normalized-record preview workflow.

Production Lines

This view must show production status by line, station, work order, and batch, including progress, WIP, freshness, exceptions, provenance, and management actions.

Required Technical Framework

Build a self-contained solution using the required technical framework below.

Candidates may use any starter repository included in the assessment package, but the solution must not depend on Celesnity repositories, services, credentials, documentation, or infrastructure that have not been provided to them.

Area	Required framework
Backend	TypeScript, Node.js 22+, NestJS 11
Persistence	PostgreSQL or MySQL
API	REST
Frontend	Next.js 16, React 19
Runtime	Docker Compose
MQTT — optional	Mosquitto